

Práctica de laboratorio: Sobrecarga de operadores en C++

Programación II

1. Presentación

En esta práctica implementará una clase `Fraction` para representar fracciones racionales. El objetivo es estudiar la sobrecarga de operadores en C++ y distinguir cuándo conviene implementar un operador como **función miembro** y cuándo conviene implementarlo como **función no miembro**.

La práctica también reforzará el uso correcto de constructores, encapsulación, métodos `const`, manejo básico de errores de entrada y mantenimiento de invariantes dentro de una clase.

2. Objetivos

Al finalizar la práctica, el estudiante será capaz de:

1. Implementar una clase en C++ con atributos privados y constructores correctos.
2. Mantener invariantes internos de una clase.
3. Sobrecargar operadores aritméticos y de comparación.
4. Distinguir entre operadores implementados como funciones miembro y operadores implementados como funciones no miembro.
5. Implementar correctamente los operadores de entrada y salida `operator>>` y `operator<<`.
6. Usar pruebas simples para verificar el comportamiento de una clase.

3. Contexto del problema

Se desea implementar una clase `Fraction` que permita trabajar con fracciones racionales de la forma `numerador/denominador`.

Por ejemplo:

```
1 Fraction a(1, 2);  
2 Fraction b(1, 3);  
3  
4 Fraction c = a + b;  
5  
6 std::cout << c << '\n';
```

La salida esperada sería:

```
1 5/6
```

La clase debe garantizar que toda fracción se mantenga siempre en un estado válido. En particular, el denominador nunca debe ser cero.

4. Archivos del proyecto

El código base del laboratorio estará organizado de la siguiente forma:

```
1 .
2 |-- Makefile
3 |-- README.md
4 |-- include
5 | '-- Fraction.h
6 |-- src
7 | '-- Fraction.cc
8 '-- tests
9   '-- main.cc
```

El archivo `Fraction.h` contendrá la declaración de la clase y de las funciones asociadas. El archivo `Fraction.cc` contendrá la implementación. El archivo `tests/main.cc` contendrá pruebas simples para verificar el funcionamiento del código.

5. Requisitos de implementación

5.1. Clase Fraction

Debe implementar una clase `Fraction` con dos atributos privados:

```
1 int numerator;
2 int denominator;
```

La clase debe ofrecer, como mínimo, los siguientes constructores y métodos:

```
1 class Fraction {
2 private:
3     int numerator;
4     int denominator;
5
6 public:
7     Fraction(int numerator, int denominator);
8     explicit Fraction(int numerator);
9
10    int num() const;
11    int den() const;
12};
```

El constructor `Fraction(int numerator, int denominator)` debe rechazar denominadores iguales a cero.

El constructor `explicit Fraction(int numerator)` debe construir una fracción entera, equivalente a `numerator/1`.

Los métodos `num()` y `den()` deben ser métodos `const`.

6. Invariante de la clase

La clase `Fraction` debe mantener el siguiente invariante:

$$\text{denominator} \neq 0$$

Opcionalmente, puede mantener también las fracciones normalizadas. Por ejemplo:

```
1 2/4 -> 1/2
2 -2/4 -> -1/2
3 2/-4 -> -1/2
```

Si decide implementar normalización, asegúrese de que el signo negativo quede en el numerador y que el denominador sea siempre positivo.

7. Sobrecarga de operadores como funciones miembro

Algunos operadores modifican directamente el objeto ubicado a la izquierda del operador. En estos casos, es natural implementarlos como **funciones miembro**.

Debe implementar los siguientes operadores como funciones miembro:

```
1 Fraction& operator+=(const Fraction& other);
2 Fraction& operator-=(const Fraction& other);
```

Por ejemplo:

```
1 Fraction a(1, 2);
2 Fraction b(1, 3);
3
4 a += b;
```

Después de ejecutar el código anterior, `a` debe representar $5/6$.

La expresión `a += b` se interpreta conceptualmente como `a.operator+=(b)`. Por esta razón, `operator+=` se implementa naturalmente como función miembro: el objeto izquierdo es el que se modifica.

Ambos operadores deben retornar `*this` para permitir expresiones encadenadas.

8. Sobrecarga de operadores como funciones no miembro

Otros operadores no modifican ninguno de sus operandos. En estos casos, normalmente conviene implementarlos como **funciones no miembro**.

Debe implementar los siguientes operadores como funciones no miembro:

```
1 Fraction operator+(Fraction a, const Fraction& b);
2 Fraction operator-(Fraction a, const Fraction& b);
3 Fraction operator*(const Fraction& a, const Fraction& b);
4 Fraction operator/(const Fraction& a, const Fraction& b);
```

La implementación de `operator+` puede reutilizar `operator+=`:

```
1 Fraction operator+(Fraction a, const Fraction& b){
2     a += b;
3     return a;
4 }
```

De manera similar, `operator-` puede reutilizar `operator-=`.

9. Operadores de comparación

Debe implementar, como mínimo, los siguientes operadores de comparación como funciones no miembro:

```
1 bool operator==(const Fraction& a, const Fraction& b);
2 bool operator!=(const Fraction& a, const Fraction& b);
3 bool operator<(const Fraction& a, const Fraction& b);
4 bool operator<=(const Fraction& a, const Fraction& b);
5 bool operator>(const Fraction& a, const Fraction& b);
6 bool operator>=(const Fraction& a, const Fraction& b);
```

Sugerencia: no es necesario implementar todos desde cero. Puede definir algunos operadores a partir de `operator<` y `operator==`.

Para comparar fracciones sin usar números reales, puede comparar mediante productos cruzados: $a/b < c/d$ si y solo si $a*d < c*b$.

10. Operador de salida `operator<<`

Debe implementar el operador de salida como función no miembro:

```
1 std::ostream& operator<<(std::ostream& os, const Fraction& f);
```

Este operador debe permitir escribir:

```
1 Fraction f(3, 4);
2
3 std::cout << f << '\n';
```

La salida debe ser 3/4.

El operador debe retornar el flujo recibido para permitir encadenar salidas:

```
1 std::cout << "f = " << f << '\n';
```

`operator<<` se implementa como función no miembro porque el operando izquierdo es un objeto de tipo `std::ostream`, por ejemplo `std::cout`.

11. Operador de entrada `operator>>`

Debe implementar el operador de entrada como función no miembro:

```
1 std::istream& operator>>(std::istream& is, Fraction& f);
```

Este operador debe leer fracciones con el formato `numerador/denominador`.

Ejemplos de entrada válida:

```
1 3/4
2 -2/5
3 10/1
```

Si la entrada es válida, `f` debe actualizarse con la fracción leída. Si la entrada es inválida, el operador debe marcar el flujo en estado de error usando `is.setstate(std::ios::failbit)`.

Casos inválidos:

```
1 3/0
2 3-4
3 abc
```

El operador no debe dejar al objeto `f` en un estado inválido. Sugerencia: primero lea los datos en variables locales. Solo construya y asigne una nueva fracción si la entrada es válida.

12. Preguntas conceptuales

Responda brevemente en el archivo `README.md` del repositorio:

1. ¿Por qué `operator+=` se implementa naturalmente como función miembro?
2. ¿Por qué `operator+` puede implementarse como función no miembro?
3. ¿Por qué `operator<<` debe implementarse como función no miembro?

4. ¿Qué significa que un método sea `const`?
5. ¿Qué invariante debe mantener la clase `Fraction`?
6. ¿Qué debe ocurrir si se intenta construir una fracción con denominador cero?
7. ¿Por qué `operator>>` no debe modificar el objeto si la entrada es inválida?

13. Pruebas mínimas

Debe incluir pruebas que verifiquen, al menos, construcción, suma, resta, multiplicación, división, comparación, salida, entrada válida y entrada inválida.

14. Restricciones

1. Los atributos de `Fraction` deben ser privados.
2. No debe usar variables globales.
3. No debe representar la fracción usando `double` o `float`.
4. No debe implementar operadores aritméticos modificando ambos operandos.
5. Los operadores `<<` y `>>` deben retornar referencias al flujo recibido.
6. Los métodos que no modifican el objeto deben declararse como `const`.
7. La clase debe impedir denominadores iguales a cero.
8. El código debe compilar sin warnings importantes usando las opciones indicadas en el `Makefile`.

15. Comandos esperados

El repositorio debe permitir compilar con:

```
1 make
```

Ejecutar las pruebas con:

```
1 make run
```

Limpiar archivos generados con:

```
1 make clean
```

16. Entregable

Debe entregar un repositorio con la estructura indicada en esta práctica. El archivo `README.md` debe incluir las respuestas a las preguntas conceptuales.

17. Criterios de evaluación

Se evaluará:

1. Correcta implementación de la clase `Fraction`.
2. Uso adecuado de constructores.
3. Mantenimiento del invariante `denominator != 0`.
4. Uso correcto de funciones miembro y funciones no miembro para la sobrecarga de operadores.
5. Correcta implementación de `operator<<` y `operator>>`.
6. Uso adecuado de `const`.
7. Claridad y orden del código.
8. Correcta compilación mediante `make`.
9. Pruebas suficientes en `tests/main.cc`.
10. Respuestas claras en el `README.md`.

18. Reflexión final

La sobrecarga de operadores debe usarse para que el código sea más natural y expresivo, no para ocultar comportamientos inesperados.

Una clase bien diseñada debe proteger sus invariantes y ofrecer operaciones coherentes con el significado del tipo que representa.